



Systèmes et Réseaux 1

Partie *Systèmes d'exploitation*

5 – Tubes de communication Unix

Tubes de communication UNIX (ou tubes « anonymes » ou « Pipes » UNIX)

- ❑ Chaque processus dispose d'un segment de données **privé**
- ❑ Deux processus quelconques ne disposent d'**aucun espace mémoire commun**

MAIS

- ❑ Il arrive que des processus produisent des informations qui devront être utilisées par d'autres processus

❑ Fichier en mémoire centrale

- ❑ Les opérations pour manipuler un tube sont **SYNTAXIQUEMENT IDENTIQUES** aux opérations pour manipuler un **fichier classique** (à l'exception de la création et de l'ouverture) : `read`, `write`, `dup`, `close`
- ❑ Deux entrées dans la table des fichiers ouverts sont associées à chaque tube

❑ Fichier de taille **limitée**

- ❑ Il ne sera pas possible d'écrire directement de très longues séquences d'octets dans un tube
- ❑ taille max d'un tube sous linux : 4096 octets (`bash -c "ulimit -a"`)

Important

❑ Accès **synchronisé**

- ❑ Les lectures et les écritures effectuées par les processus devront respecter certaines **règles quant à l'ordre d'exécution** et aux conditions dans lesquelles elles devront être effectuées

❑ Fichier ayant des **lectures destructrices**

- ❑ A l'inverse d'une lecture classique dans un fichier, une lecture dans un tube entraîne la **suppression** des informations lues (Il s'agit d'une extraction)

❑ Via la primitive pipe

```
int pipe (int tube[2]);  
/* En retour : */  
/* tube[0] est ouvert en lecture */  
/* tube[1] est ouvert en écriture */
```

❑ Ce tube est connu et sera utilisable par le processus appelant ainsi que tous les processus de sa **descendance** (comme tous les fichiers ouverts avant l'appel de fork)

❑ Sémantique

Important

- Ouverture de **deux descriptifs de fichiers** dans la FDT
 - ❑ Une entrée correspond à une ouverture du tube en **lecture**
 - ❑ L'autre entrée correspond à l'ouverture du tube en **écriture**
- Les deux entrées sont mémorisées dans les deux éléments du tableau paramètre
- pipe retourne la valeur -1 si la création du tube échoue

Exemple de création d'un tube de communication

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    int tube[2];
    if (pipe(tube) == -1)
    {
        perror("Échec création tube");
        exit(1);
    }
    int pid=fork() ;
    if (pid==-1)
    {
        perror("Échec création fils");
        exit(2);
    }
    if (pid==0) // Contexte du fils
    {
        printf("Code du fils \n");
        ....
        exit(0);
    }
    .... // Contexte du père
    exit(0);
}
```

Tableau à 2 entrées pour récupérer
les 2 descripteurs
tube[0] : ouvert en lecture
tube[1] : ouvert en écriture

La création du tube peut échouer,
en tenir compte

Le fils hérite de la valeur
des descripteurs du tube,
ouverts en lecture et en écriture

□ Un lecteur lit dans un tube en utilisant la primitive read (cf. chapitre 3 - Primitives E/S)

□ **Sémantique spécifique** au tube

➤ Cas 1 : Les N octets demandés sont présents dans le tube

- Les N octets sont retirés et transmis au processus appelant dans la zone définie par *adresse*
- read **retourne** le nombre N d'octets transmis

➤ Cas 2 : Le tube n'est pas vide mais contient moins des N octets demandés

- Les octets présents sont retirés et transmis au processus appelant dans la zone définie par *adresse*
- read **retourne** le nombre d'octets effectivement présents et transmis

➤ Cas 3 : Le tube est **vide**

- Le processus appelant est **bloqué jusqu'à** ce que l'une des deux conditions suivantes soit vérifiée
 - Il y a un nombre non nul d'octets présents dans le tube (le tube n'est plus vide)
 - Le tube n'est plus ouvert en écriture par **aucun** processus

```
#include <unistd.h>

int read(
    int fd,
    void *adresse,
    unsigned nombreDemande) ;
```

**Très
Important**

- Un rédacteur écrit dans un tube en utilisant la primitive `write` (cf. chapitre 3 - Primitives E/S)

Très
Important

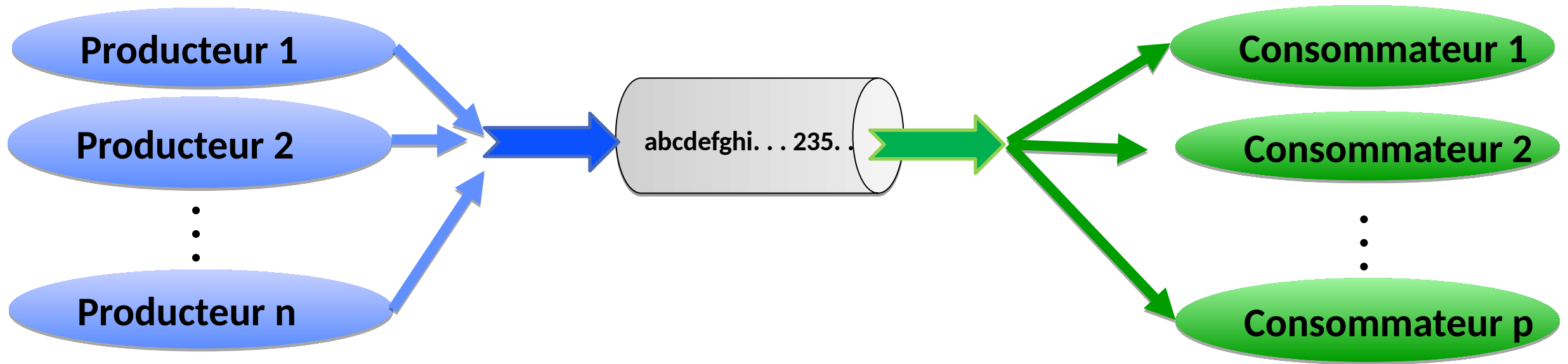
```
#include <unistd.h>

int write(
    int fd,
    void *adresse,
    unsigned nombreTransmis);
```

- Sémantique spécifique au tube

- L'opération `write` ne se termine que lorsque les *nombreTransmis* octets transmis par le processus appelant dans la zone désignée par *adresse* ont été écrits dans le tube
 - Le transfert est réalisé au fur et à mesure que la place se libère dans le tube
 - Lorsque le tube est plein, le processus appelant est bloqué jusqu'à ce qu'un lecteur libère de la place par appel à `read`
- L'écriture dans un tube qui n'est plus ouvert en lecture génère une erreur. Le signal « SIGPIPE » interrompt le processus, s'il n'est pas géré dans le code (voir le cours sur les signaux en SR2)
- Si un incident est détecté lors d'un `write` (réception d'un signal, capacité maximale du support atteinte, etc.), l'écriture est interrompue
- `write` retourne le nombre d'octets effectivement écrits ou -1 en cas d'erreur

- Certains processus sont amenés à écrire dans un tube : **producteurs**
- Certains processus sont amenés à lire dans un tube : **consommateurs**



□ Principe de la synchronisation

- Une **opération** qui n'est **pas possible** est **bloquante** pour le processus **appelant**
- La **lecture** est bloquante si le **tube** est **vide**
- L'**écriture** est bloquante si le **tube** est **plein**

- Il est préférable d'orienter les tubes entre les deux familles de processus
 - L'utilisation d'un même tube en lecture ET en écriture, par un même processus, est dangereuse
 - Elle peut conduire à un interblocage (*deadlock*) et doit être effectuée avec précaution
 - Chaque processus devra donc fermer l'entrée du tube non utilisée
 - **Attention**
 - Il est important de toujours tester la réponse de la primitive **pipe** afin de vérifier que le tube souhaité a correctement été créé
 - La fermeture de l'entrée en écriture est obligatoire, sinon cela a des conséquences sur le déblocage des lecteurs en attente
 - La fermeture de l'entrée en lecture est optionnelle mais elle a l'avantage de libérer une entrée de la FDT dont le nombre d'entrées est limité
- ➔ **Toujours fermer une entrée d'un tube dès lors qu'elle n'est plus utilisée**

Entre deux processus père et fils

➤ Création du tube

Appel de la primitive « pipe »

write(T[1]...);

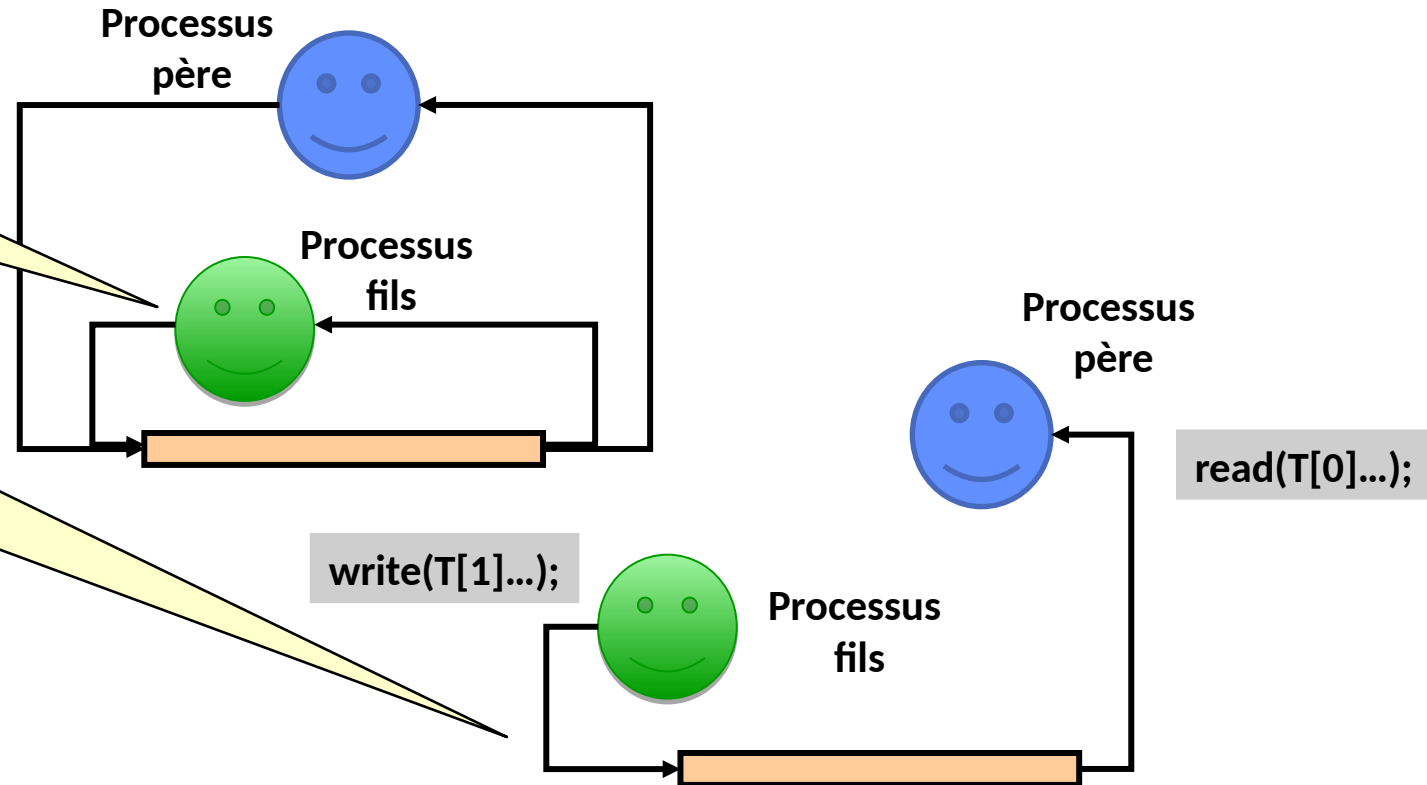
read(T[0]...);

➤ Création du fils

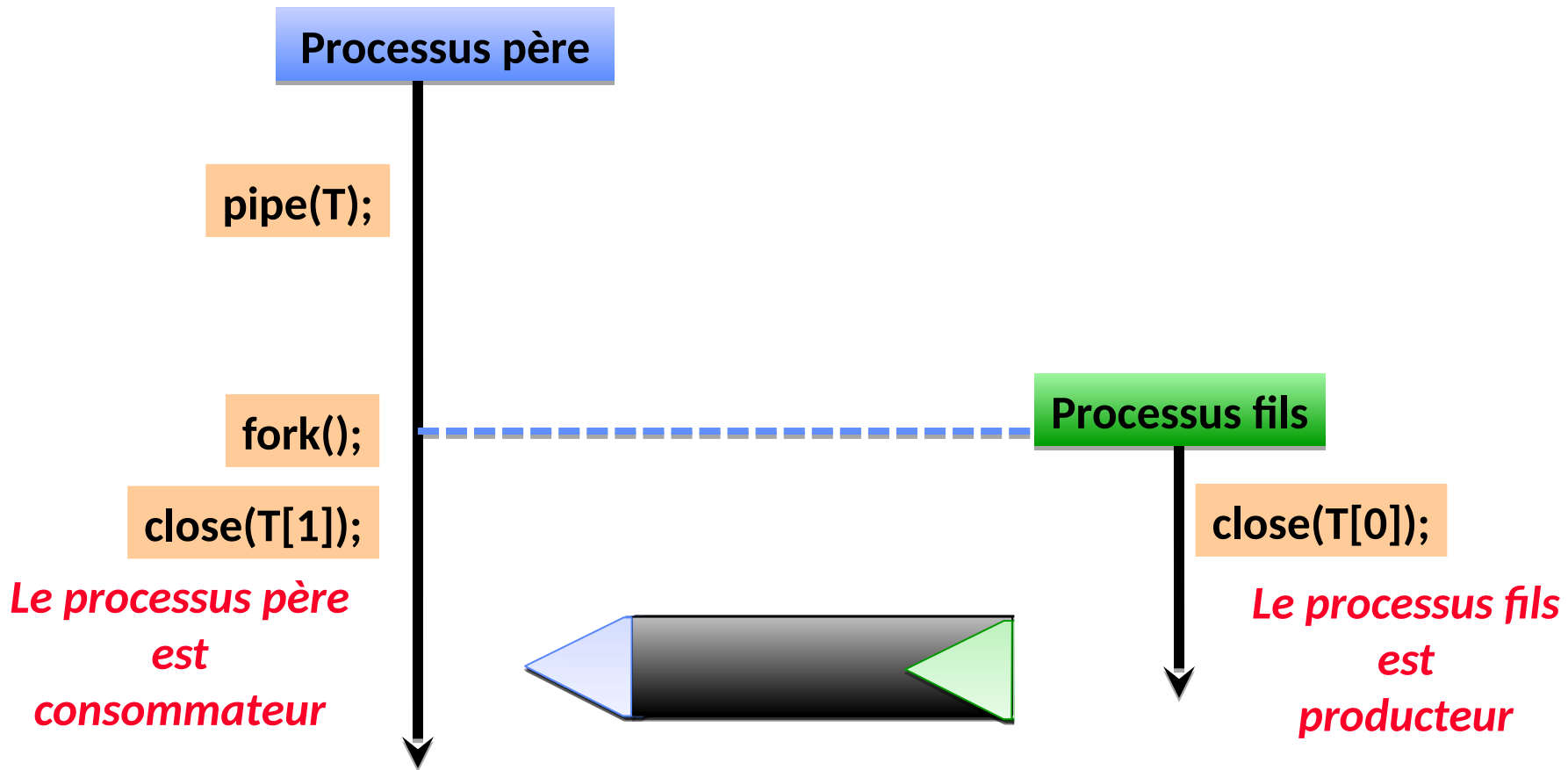
Exécution de la primitive « fork »

➤ Orientation du tube

Fermetures de l'entrée non utilisée
par chacun des processus



Relation producteur/consommateur – Autre illustration



```
char msg[] = "Tu as le bonjour de ton fils ";
int main()
{
    int tube[2];
    char car;
    printf("*** Début du travail processus principal\n");
    if (pipe(tube) == -1)
    {
        perror("Échec création tube");
        exit(1);
    }
    int pid = fork() ;
    if (pid == -1)
    {
        perror("Échec création fils");
        exit(2);
    }
    if (pid == 0)
    {
        printf("FILS démarre \n");
        close(tube[0]); /* Fils est rédacteur */
        write(tube[1], (void *)msg, sizeof(char) *( strlen(msg) + 1));
        close(tube[1]);
        printf("FILS se termine\n");
        exit(0);
    }
}
```

Déclaration du tableau nécessaire à la création du tube (une instance par processus)

Création du tube et test de bon fonctionnement

Le fils hérite des descripteurs ouverts en lecture & écriture

Le fils ne lira jamais dans le tube, il ferme l'entrée ouverte en lecture

Écriture dans le tube, en utilisant l'entrée ouvert en écriture

Le fils a fini d'écrire dans le tube, il ferme l'entrée ouverte en écriture

```
char msg[] = "Tu as le bonjour de ton fils ";
int main()
{
    int tube[2];
    char car;
    printf("*** Début du travail processus principal\n");
    if (pipe(tube) == -1)
    {
        perror("Échec création tube");
        exit(1);
    }
    int pid = fork() ;
    if (pid == -1)
    {
        perror("Échec création fils");
        exit(2);
    }
    if (pid == 0)
    {
        printf("FILS démarre \n");
        close(tube[0]); /* Fils est rédacteur */
        write(tube[1], (void *)msg, sizeof(char) *( strlen(msg) + 1));
        close(tube[1]);
        printf("FILS se termine\n");
        exit(0);
    }
}
```

Le père n'écrira jamais dans le tube,
il ferme l'entrée ouverte en écriture

```
close(tube[1]); /* Père est lecteur */
printf("*** Le père reçoit \n");
while (read (tube[0], (void *)&car, sizeof(char)))
    printf("%c", car);

close(tube[0]);
printf(" \n");
printf("*** Fin du travail du père \n");
exit(0);
}
```

Lecture dans le tube,
en utilisant le descripteur
ouvert en lecture
[Lecture coûteuse
car caractère par caractère]

Le père a fini de lire dans le tube,
il ferme l'entrée ouverte en
lecture

Tubes – Exemple : Production/Consommation – Exécution

```

char msg[] = "Tu as le bonjour de ton fils ";
int main()
{
  int tube[2];
  char car;
  printf("*** Début du travail processus principal\n");
  if (pipe(tube) == -1)
  {
    perror("Échec création tube");
    exit(1);
  }
  int pid = fork() ;
  if (pid == -1)
  {
    perror("Échec création fils");
    exit(2);
  }
  if (pid == 0)
  {
    printf("Fils démarre \n");
    close(tube[0]);      /* Fils est rédacteur */
    write(tube[1], (void *)msg, sizeof(char) *( strlen(msg) + 1));
    close(tube[1]);
    printf("Fils se termine\n");
    exit(0);
  }

```

```

close(tube[1]); /* Père est lecteur
printf("*** Le père reçoit \n");
while (read (tube[0], (void *)&car,
  printf("%c", car);

close(tube[0]);
printf(" \n");
printf("*** Fin du travail du père \n");
exit(0);
}

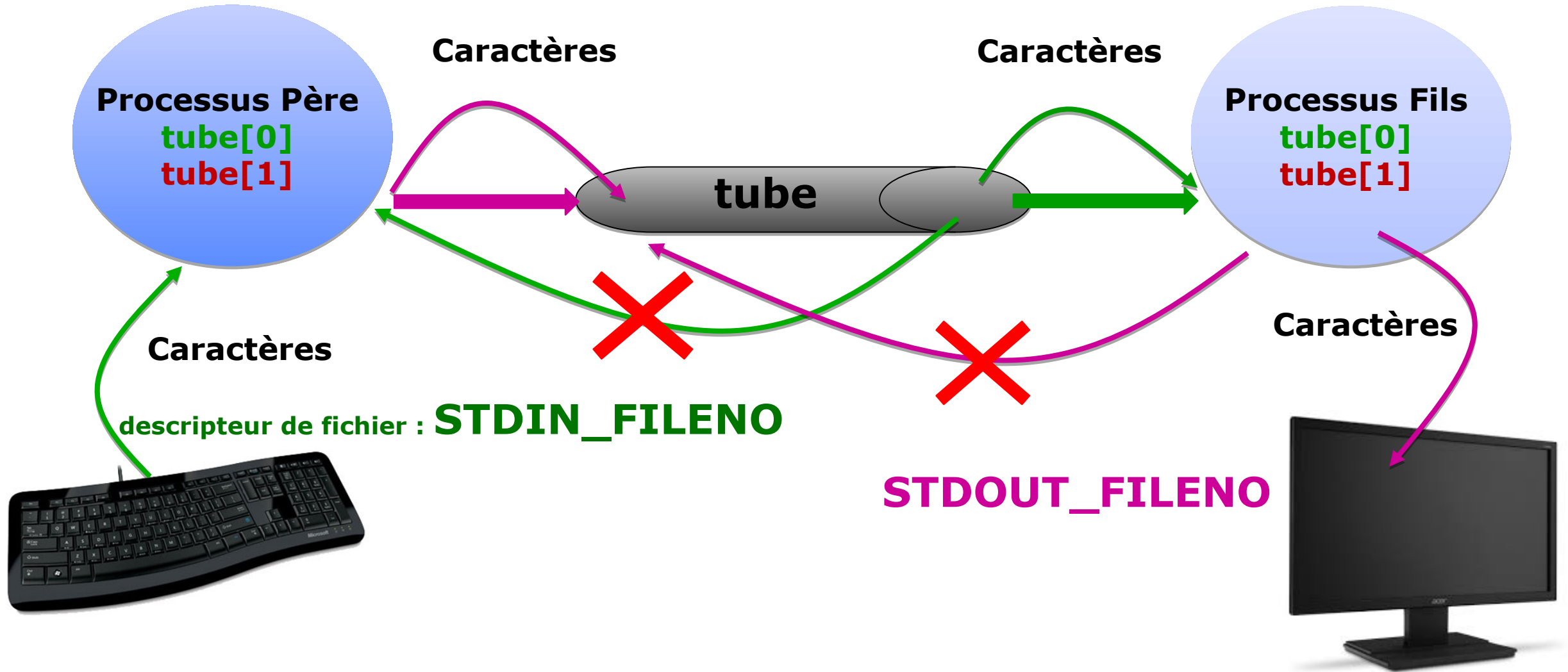
```

```

$ ./ex_pipe
*** Début du travail processus principal
*** Le père reçoit
Fils démarre
Fils se termine
Tu as le bonjour de ton fils
*** Fin du travail du père
$

```

- ❑ Écrire une application dans laquelle le processus père lit les caractères au clavier et le processus fils affiche ces mêmes caractères à l'écran de l'utilisateur
- ❑ Étape 1 : Fournir un **schéma** indiquant la manière dont le tube est utilisé par les différents processus
- ❑ Étape 2 : Écrire le code de l'application



□ Le paramètre commande précise l'action à réaliser sur l'entrée de la table précisée par le descripteur de fichier ouvert `fd`

□ Les paramètres supplémentaires vont dépendre de la commande spécifiée

➤ `F_GETFL` : Permet de récupérer les valeurs des flags associés au descripteur de fichier ouvert

➤ `F_SETFL` : Permet de réécrire les valeurs des flags associés au descripteur de fichier ouvert

□ Pour modifier le flag de non blocage :

➤ `F = fcntl(fd, F_GETFL);` `/* Récupération des flags */`

➤ `F = F | O_NONBLOCK;` `/* Positionnement du flag de non blocage */`

➤ `fcntl(fd, F_SETFL, F);` `/* Réécriture */`

➤ Avec les tests associés sur les erreurs possibles...

```
#include <unistd.h>
#include <fcntl.h>

int fcntl(
    int fd,
    int commande,
    ... );
```

Lecture non bloquante

- **Sémantique de la primitive read après avoir positionné le flag O_NONBLOCK :**
 - ☐ Si des processus disposent d'un accès en écriture sur le tube et que le tube est vide, -1 est retourné par la primitive read et errno contient la valeur EAGAIN (sinon, le processus resterait bloqué dans la lecture)

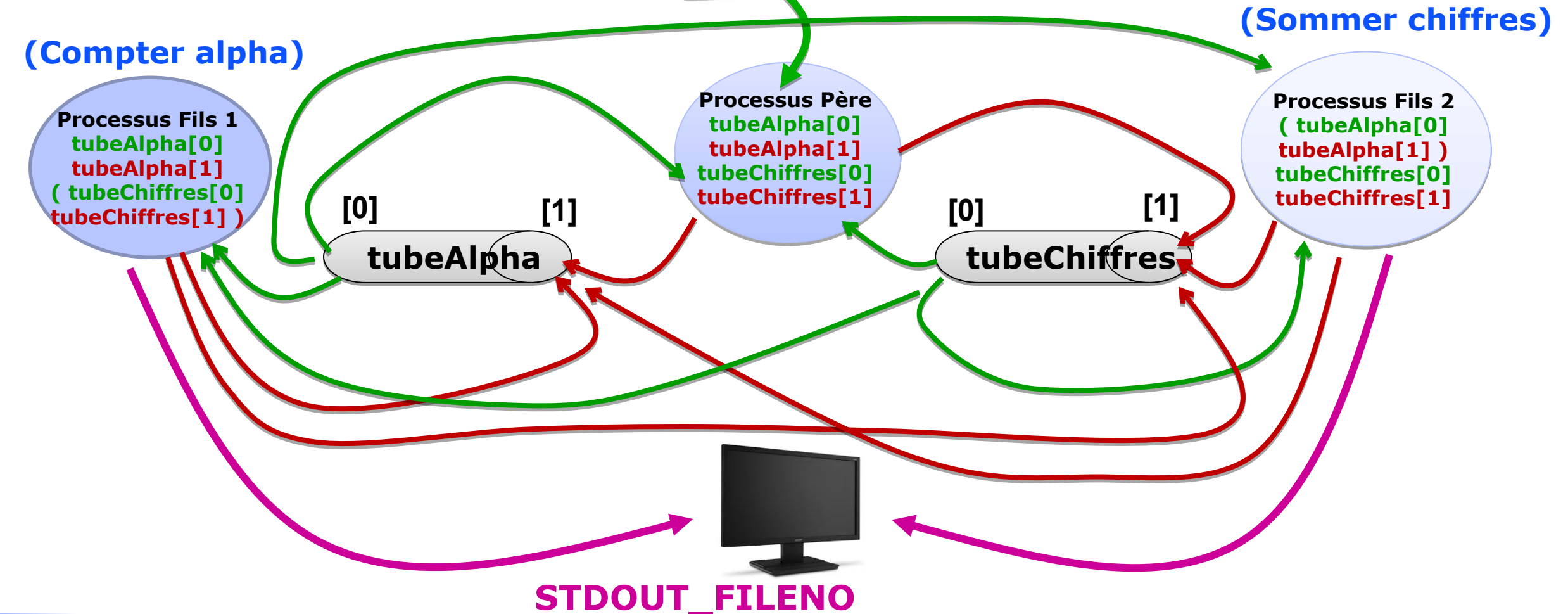
Écriture non bloquante

- **Sémantique de la primitive write après avoir positionné le flag O_NONBLOCK :**
 - ☐ La primitive write écrira 0 ou N octets selon les cas, mais le processus ne restera pas bloqué (consulter man pour plus de détails)

- On se propose d'écrire un programme C dans lequel interviennent trois processus :**
 - Le processus *père* reçoit sur son entrée standard un flot de caractères. Un caractère alphabétique est transmis au fils *Alphabétiques*. Un caractère chiffre est transmis au fils *Chiffres*. Les autres caractères sont ignorés. Lorsque la fin du flot est détectée (fin de fichier au clavier ou fin d'un fichier disque), le processus père attend la terminaison de ses fils avant de se terminer lui-même
 - Le processus *Alphabétiques* reçoit de son père des caractères alphabétiques. Il doit compter le nombre d'occurrences de chaque caractère alphabétique indépendamment de la casse des caractères (3 caractères 'A' et 2 caractères 'a' donnent 5 caractères 'a'). Lorsque la fin du flot est détectée, le processus *Alphabétiques* affiche ses statistiques, puis se termine
 - Le processus *Chiffres* reçoit de son père des chiffres. Il doit calculer la somme des chiffres reçus. Lorsque la fin du flot est détectée, le processus *Chiffres* affiche la somme calculée, puis se termine
- On pourra utiliser les fonctions suivantes de la bibliothèque `<ctype.h>` :**
 - `int isalpha(char C) ; /* 1 si alphabétique, 0 sinon */`
 - `int isdigit(char C) ; /* 1 si numérique, 0 sinon */`
 - `char tolower(char C) ; /* si C est alphabétique, retourne la minuscule correspondante sinon retourne C */`
- Proposer un schéma de communication entre les processus mettant en jeu un(des) tube(s) de communication avant d'écrire l'application**

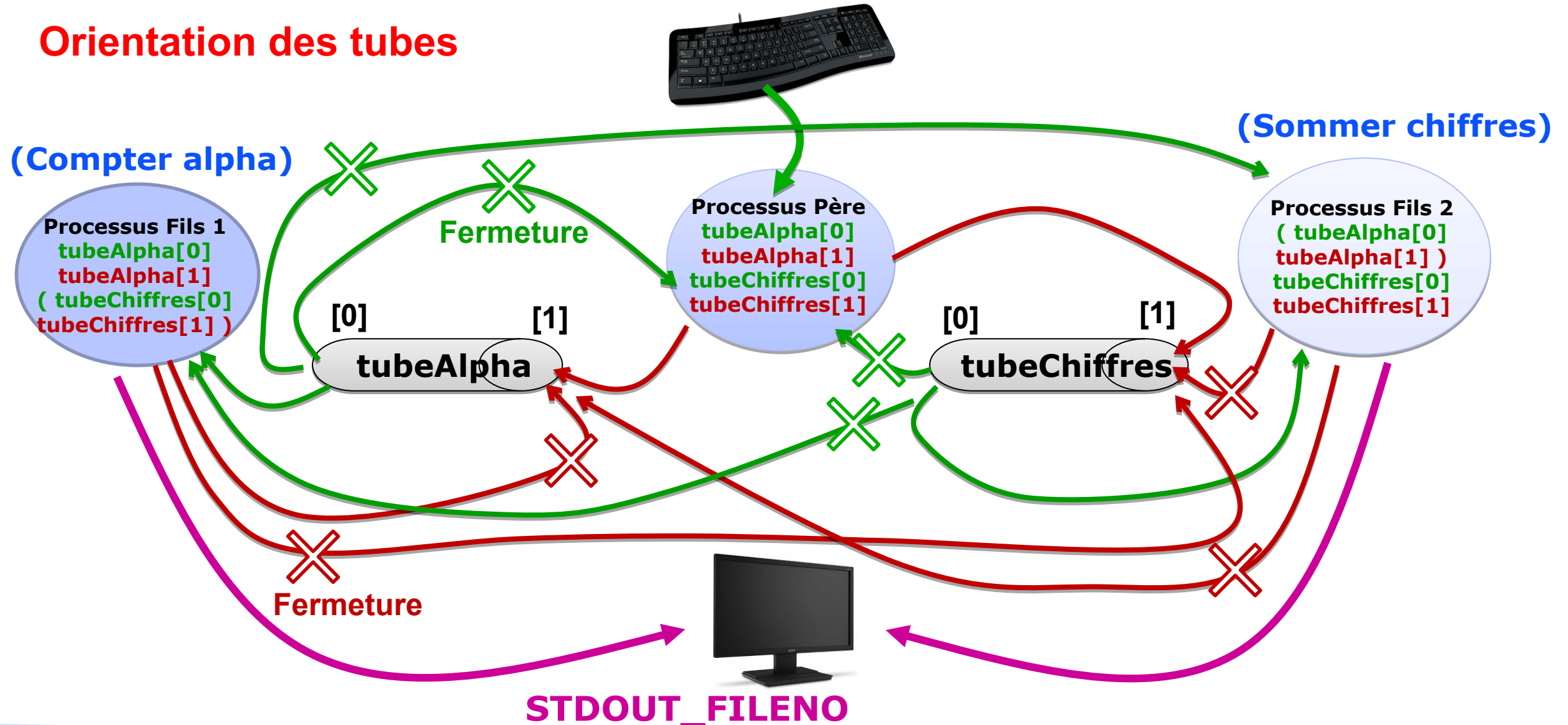
descripteur de fichier : **STDIN_FILENO**

Le père crée 2 tubes puis 2 fils



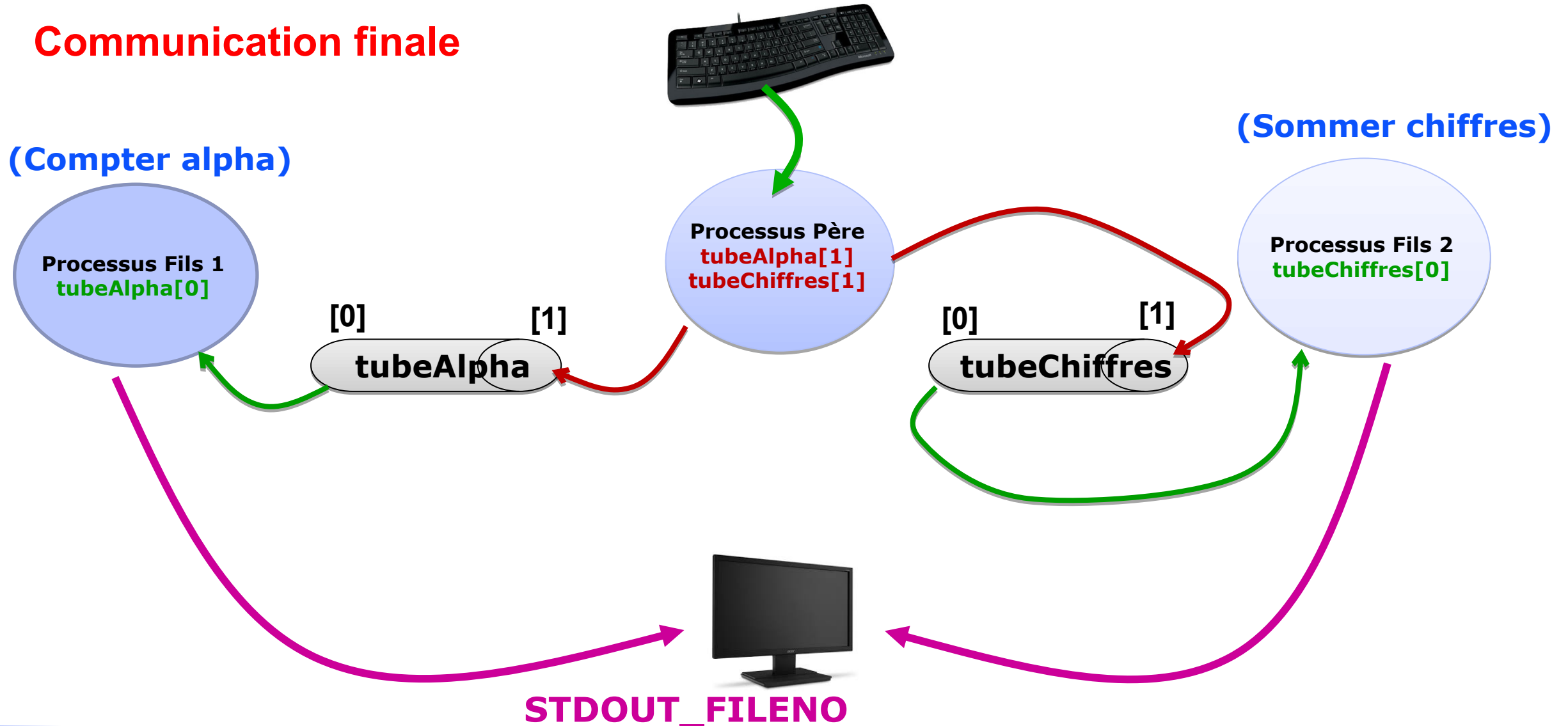
descripteur de fichier : **STDIN_FILENO**

Orientation des tubes



descripteur de fichier : **STDIN_FILENO**

Communication finale

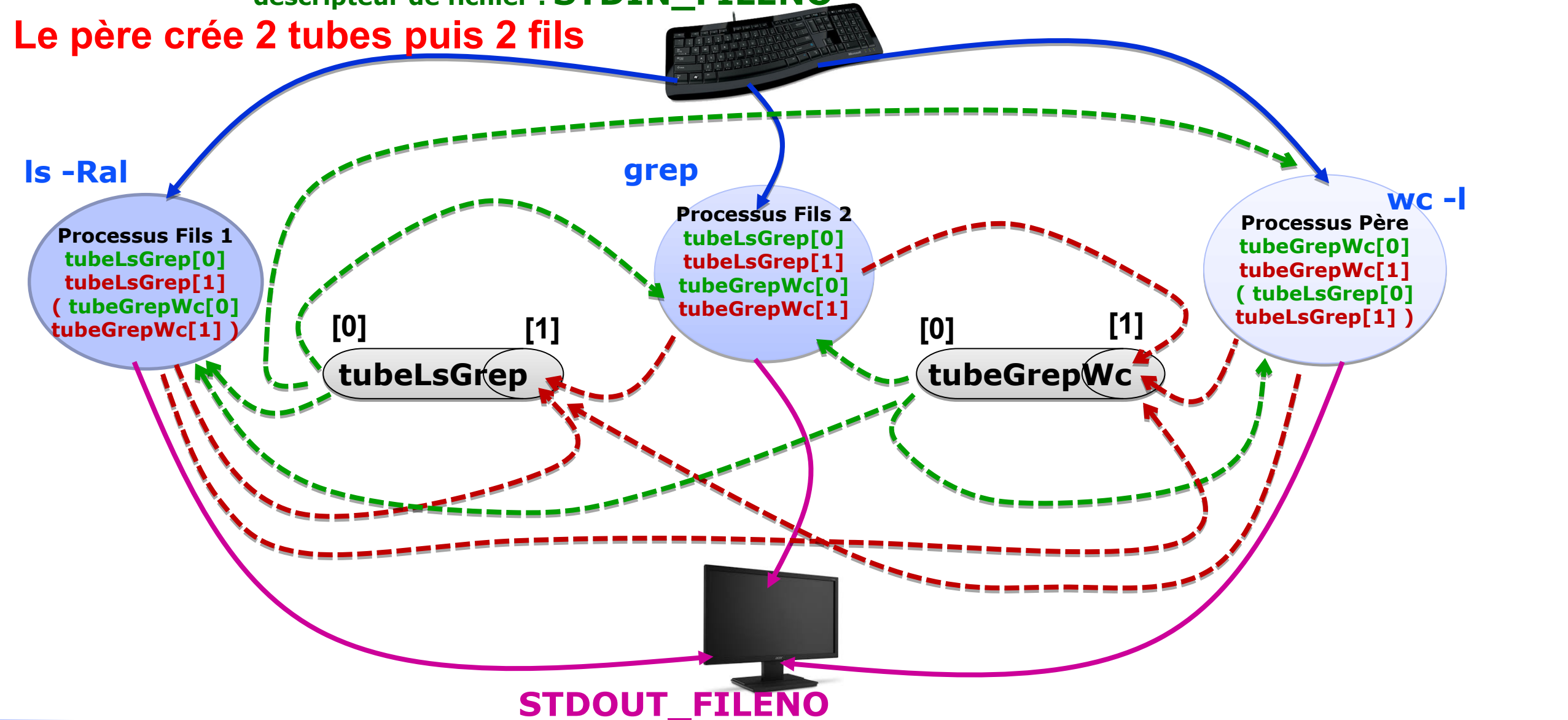


- ❑ Programmer la commande
compter HIERARCHIE UTILISATEUR
qui réalise le traitement suivant:
`ls -Ral HIERARCHIE | grep UTILISATEUR | wc -l`
- ❑ Étape 1 : Fournir un **schéma** indiquant la manière dont le(s) tube(s) est(sont) utilisé(s) par les différents processus
- ❑ Étape 2 : Écrire le code de l'application

Tubes – Exercice 3 – Mise en œuvre de la communication

descripteur de fichier : **STDIN_FILENO**

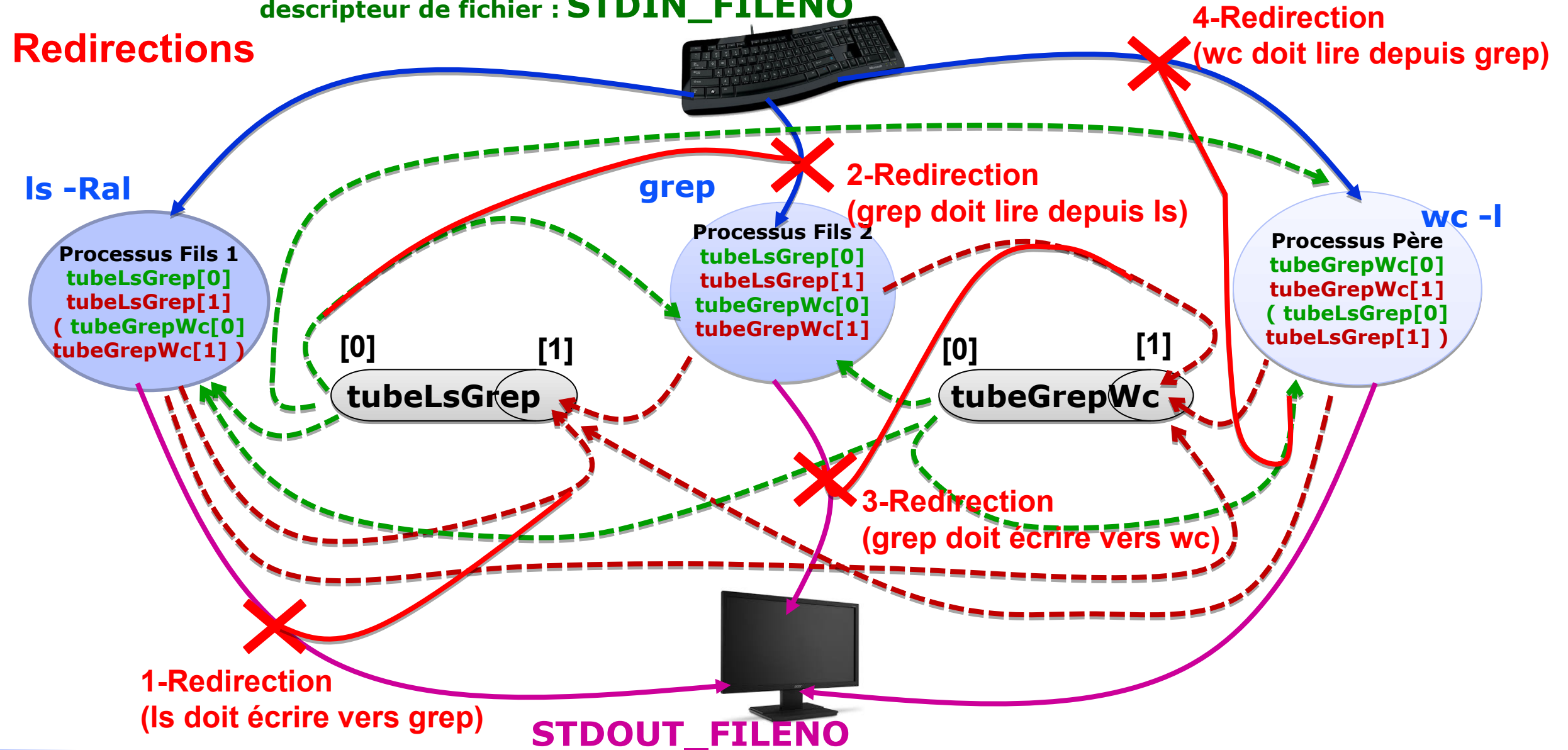
Le père crée 2 tubes puis 2 fils



Exercice 3 – Mise en œuvre de la communication

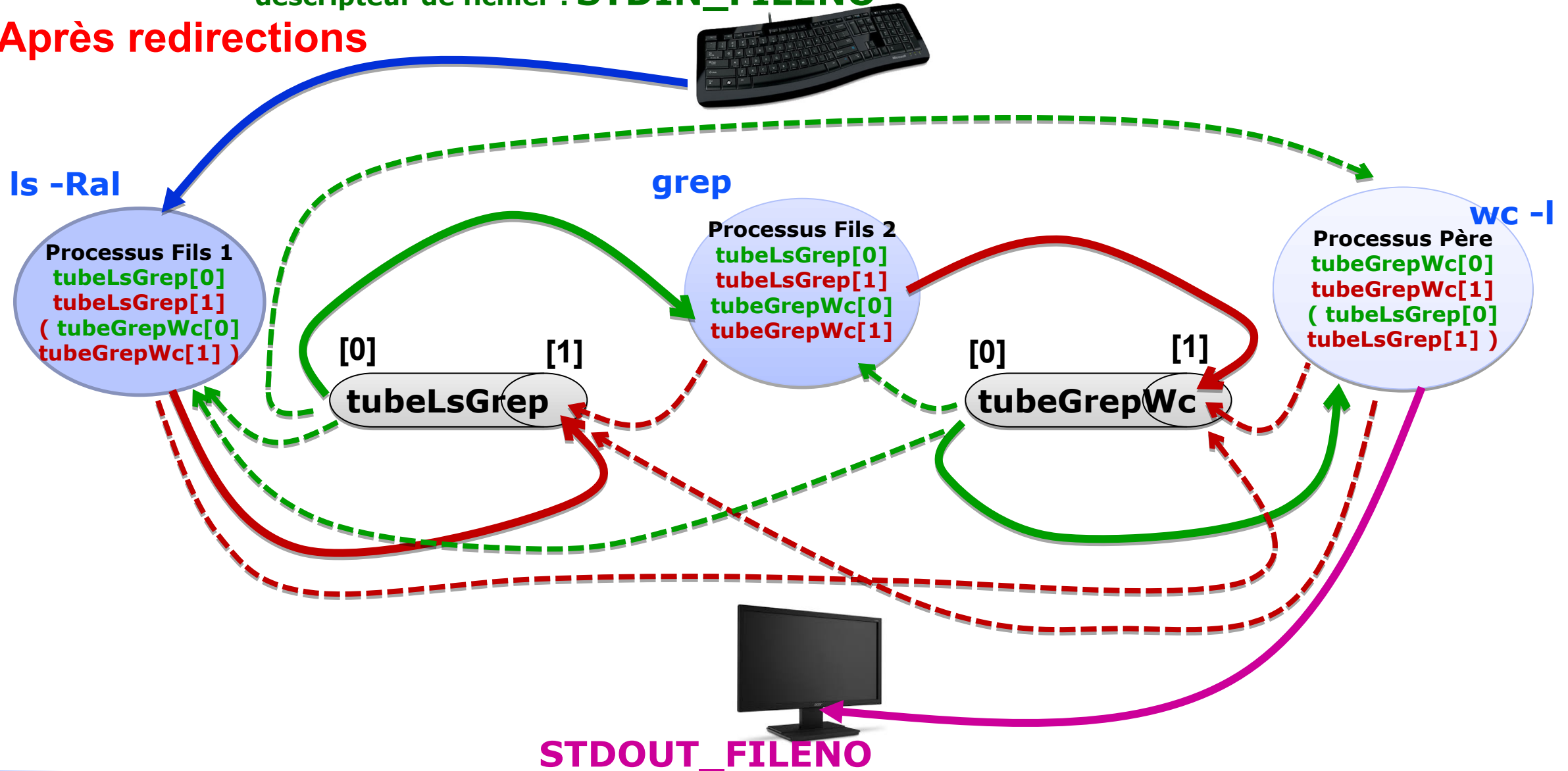
descripteur de fichier : **STDIN_FILENO**

Redirections



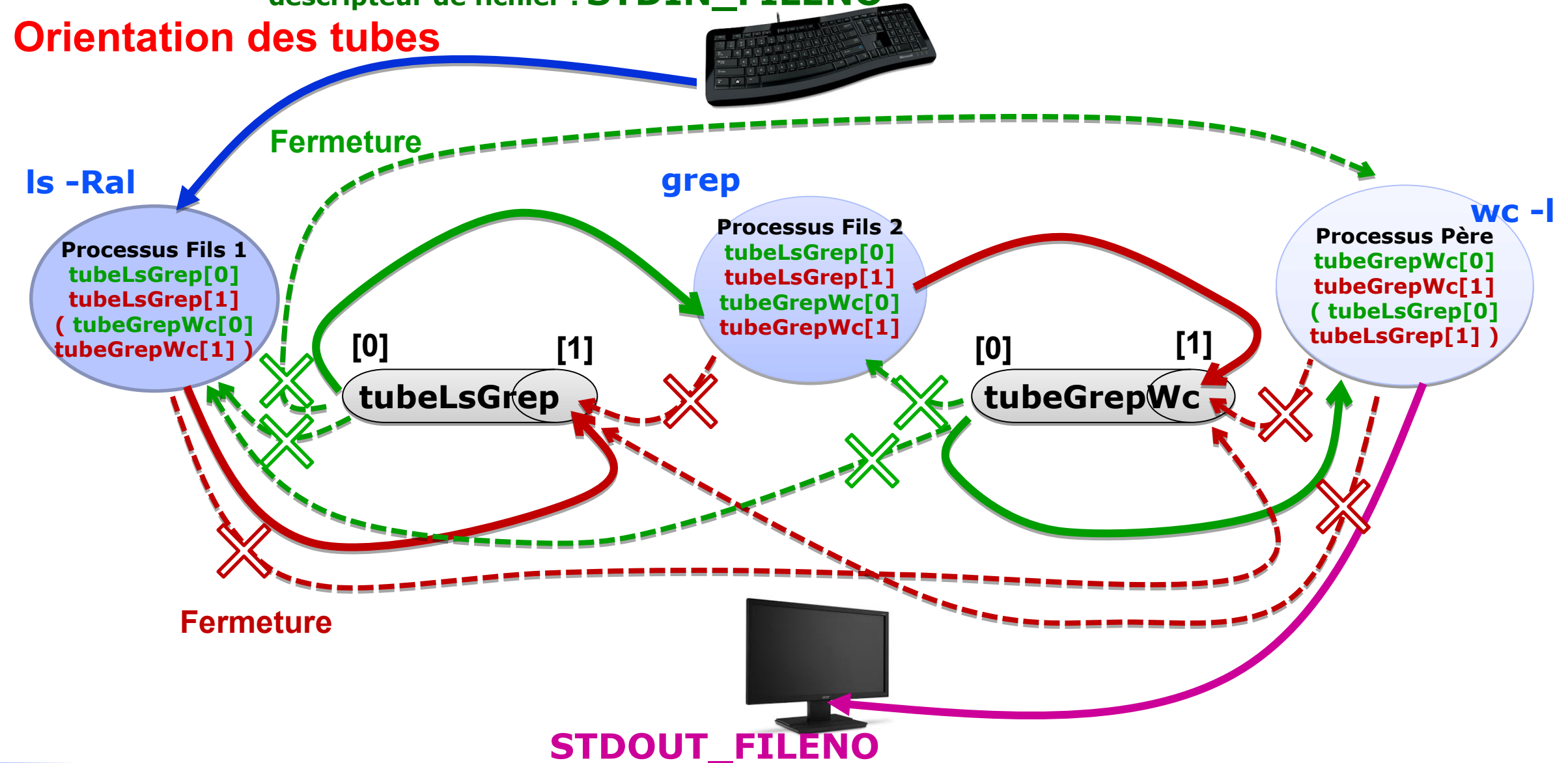
descripteur de fichier : **STDIN_FILENO**

Après redirections



descripteur de fichier : **STDIN_FILENO**

Orientation des tubes



Exercice 3 – Mise en œuvre de la communication

descripteur de fichier : **STDIN_FILENO**

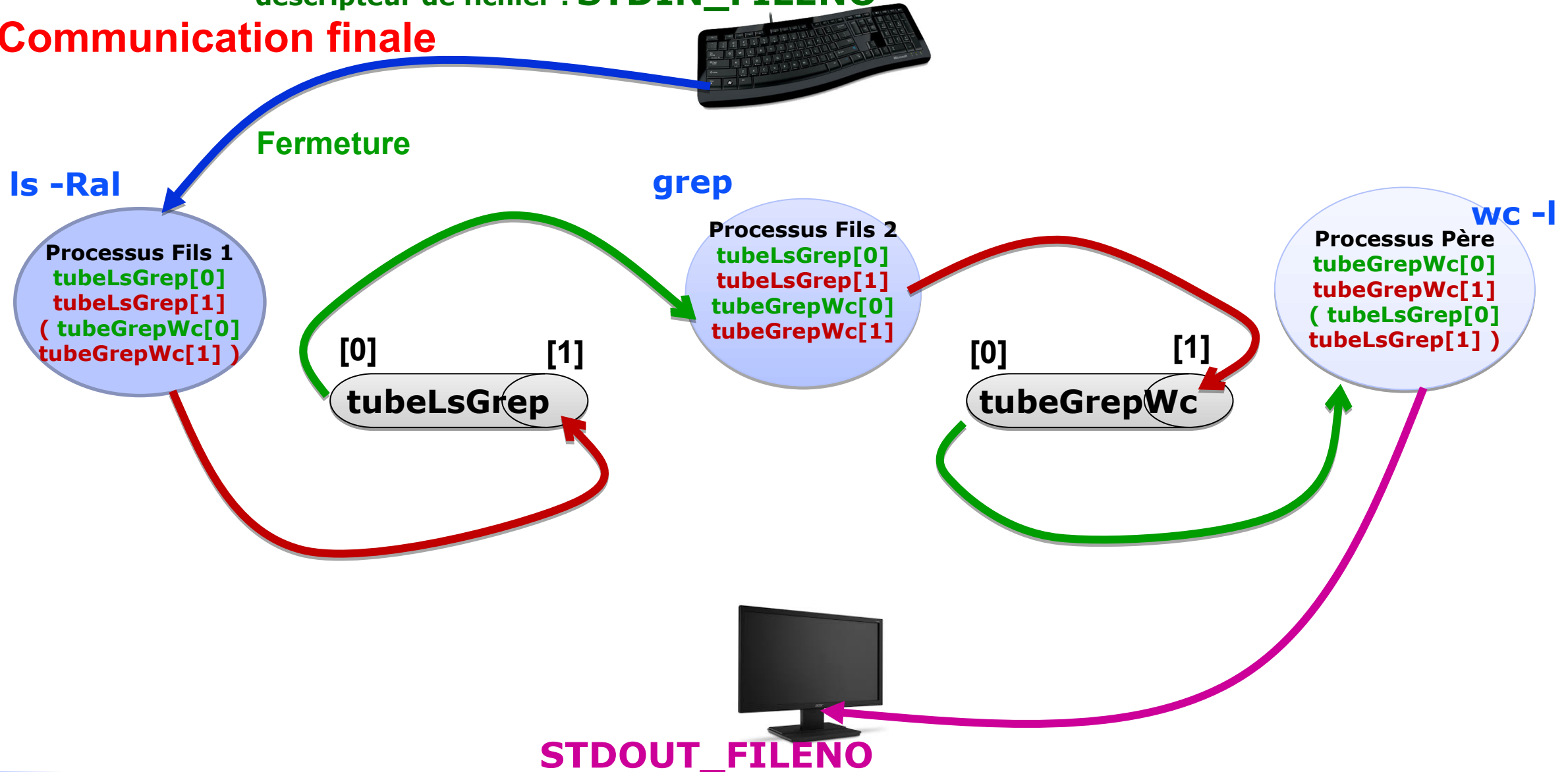
Communication finale

Fermeture

ls -Ral

grep

wc -l



STDOUT_FILENO

Être plus futé...

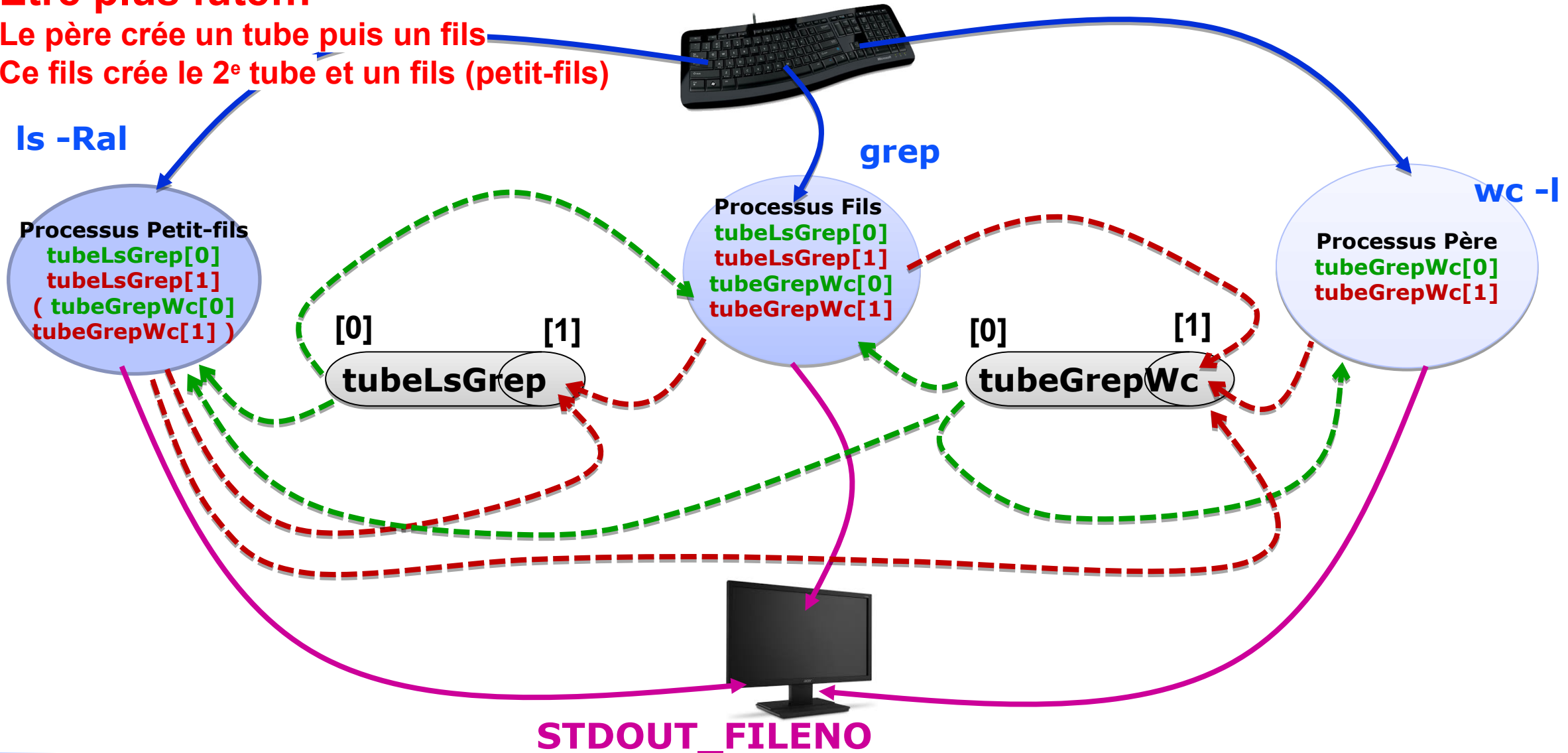
descripteur de fichier : **STDIN_FILENO**

Le père crée un tube puis un fils
Ce fils crée le 2^e tube et un fils (petit-fils)

ls -Ral

grep

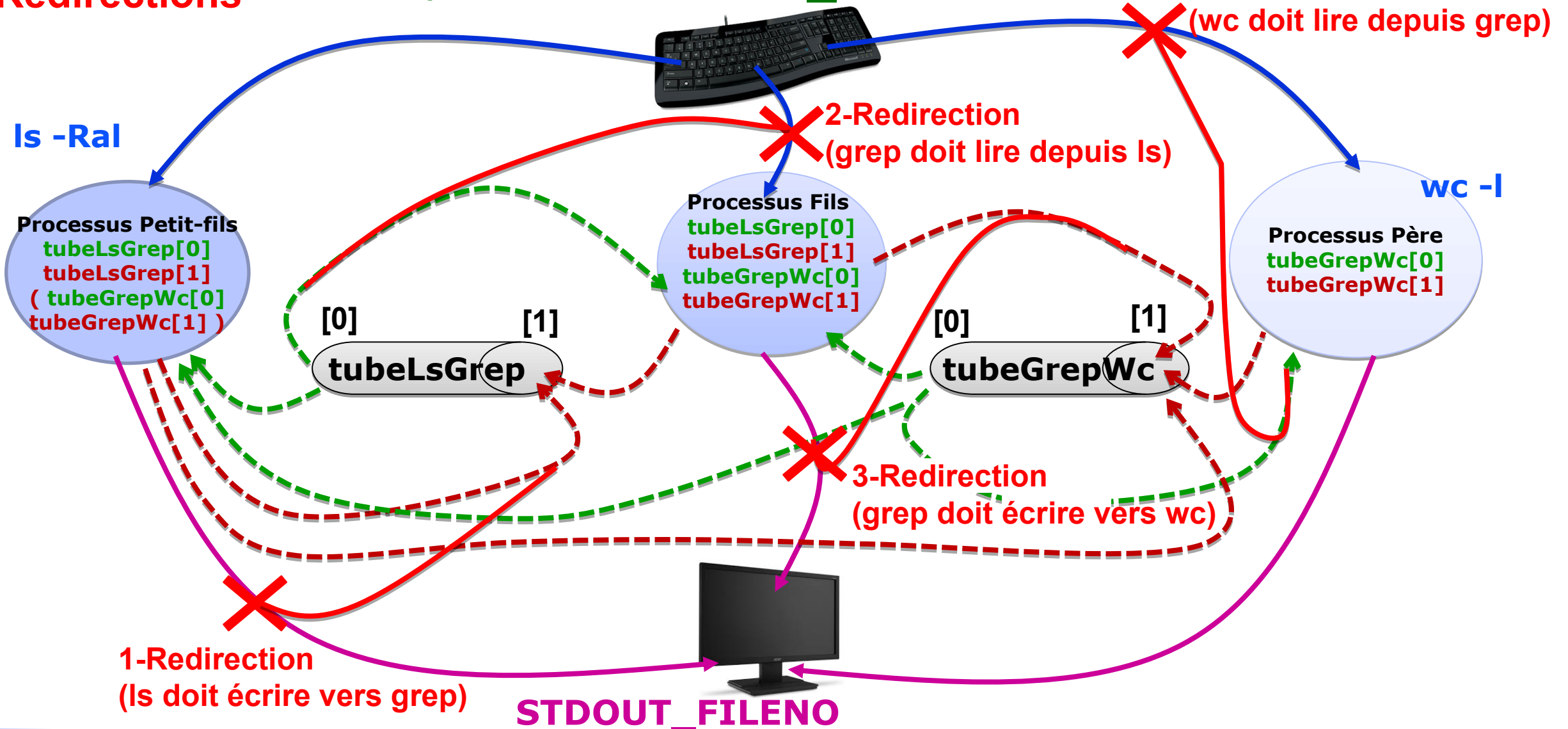
wc -l



STDOUT_FILENO

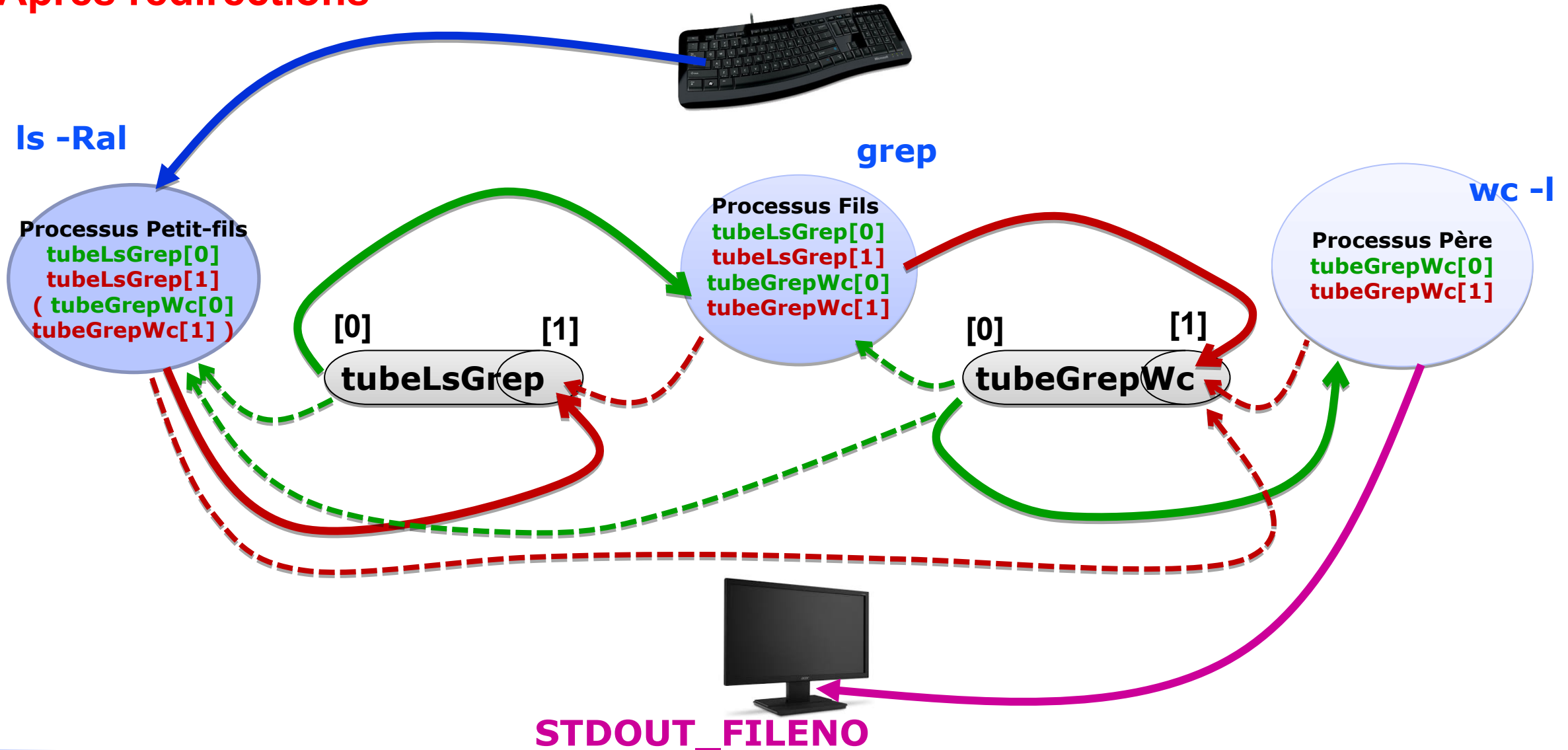
Redirections

descripteur de fichier : **STDIN_FILENO**

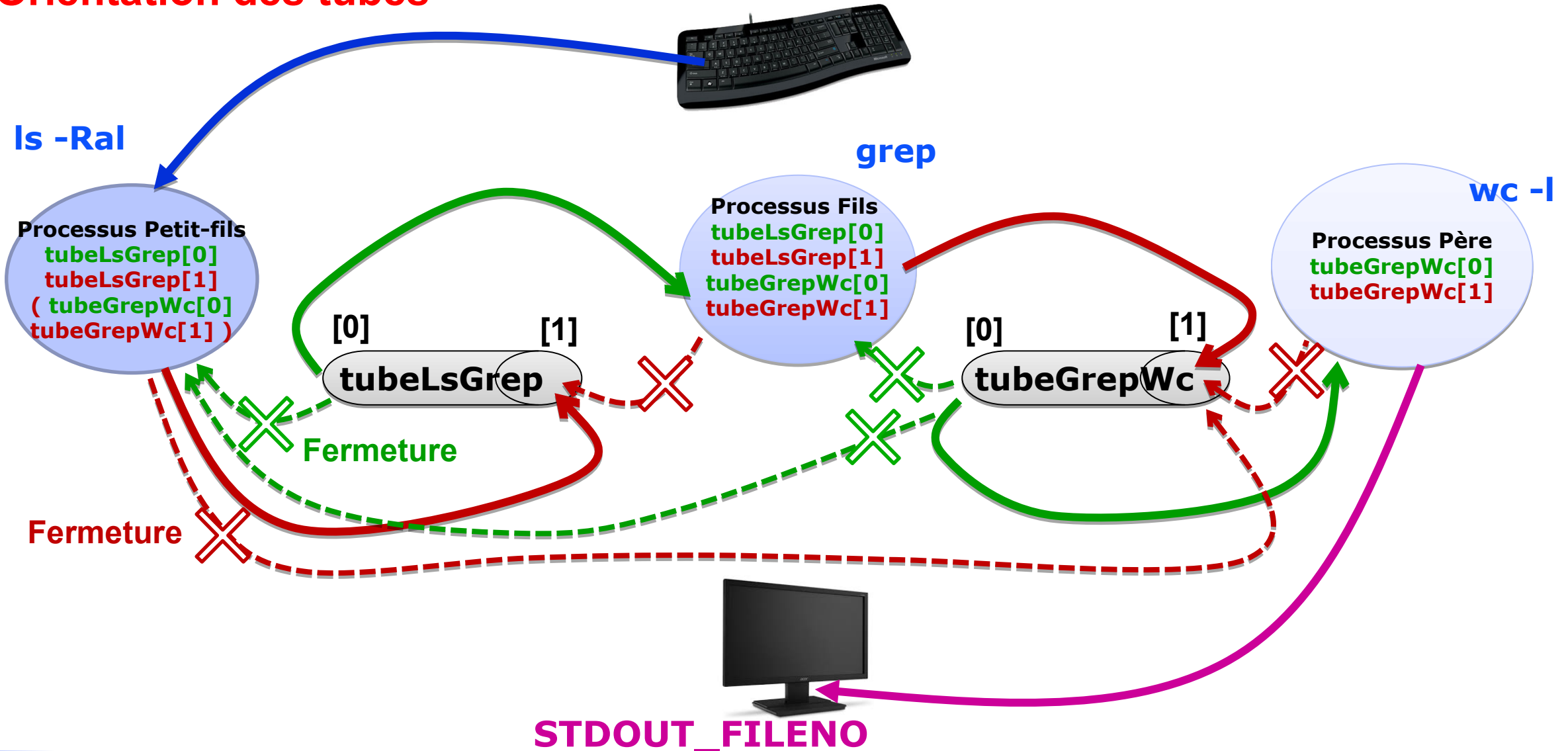


Après redirections

descripteur de fichier : **STDIN_FILENO**

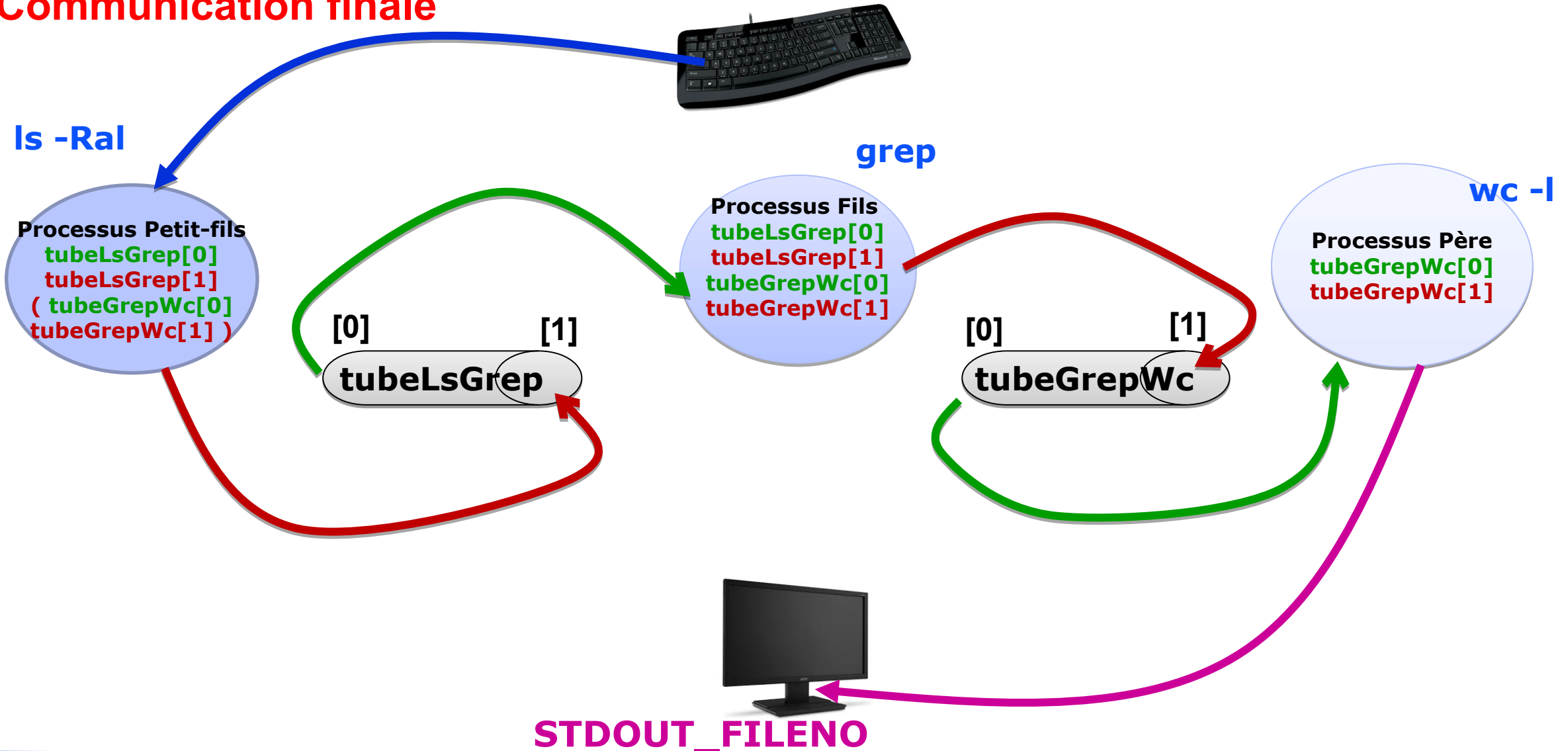


Orientation des tubes descripteur de fichier : **STDIN_FILENO**



descripteur de fichier : **STDIN_FILENO**

Communication finale



STDOUT_FILENO

Tubes nommés

- ❑ **Les tubes nommés associent les propriétés des tubes avec certaines propriétés des fichiers**
 - Ils sont de taille **limitée**
 - Les lectures et écritures sont effectuées suivant un ordre **FIFO**
 - Les lectures sont **destructrices**
 - Ils sont utilisables par des processus qui ne sont **pas obligatoirement de la même** famille

□ Primitive mkfifo

- Crée un tube de nom spécifié
- mode précise les droits d'accès aux trois classes d'utilisateurs
- mkfifo retourne 0 en cas de succès, -1 en cas d'erreur (utiliser perror() dans ce cas)

```
#include <unistd.h>
#include <sys/stat.h>

int  mkfifo(
    const char *nom,
    mode_t      mode) ;
```

□ Utilisation

- Tout processus qui veut utiliser un tube nommé doit connaître son nom
- Un processus doit ouvrir le tube avant de l'utiliser
- Un processus qui essaie d'ouvrir un tube nommé en lecture sera **bloqué** jusqu'à ce qu'un autre processus ouvre ce même tube en écriture
- et vice-versa

Tubes nommés – Exemple de création

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/stat.h>

```

```

int main(int argc, char * argv[])
{
    printf("*** Début du processus création tube nommé\n");
    if (argc!=2)
    {
        fprintf(stderr,"Usage : %s nom_tube\n",argv[0]);
        exit(1);
    }
    if (mkfifo(argv[1], 0600) == -1)
    {
        perror(argv[1]);
        exit(2);
    }
    printf("Le tube nommé %s a été créé \n", argv[1]);
    exit(0);
}

```

Création du tube nommé
Son nom est en paramètre
de la fonction main

Les droits d'accès précisent lecture
et écriture pour le propriétaire seul
Ceci est vérifié après exécution

```

$ ./ex_mkfifo TN
*** Début du processus création tube nommé
Le tube nommé TN a été créé
$ ls -l
total 384
prw----- 1 lovelace lovelace 0 Apr 7 17:34 TN
-rwxr-xr-x 1 lovelace lovelace 17380 Apr 7 09:37 ex_exec
-rwxr-xr-x 1 lovelace lovelace 17268 Apr 6 16:46 ex_fork
-rw-r--r-- 1 lovelace lovelace 902 Apr 7 14:13 pipe.c
-rw-r--r-- 1 lovelace lovelace 490 Apr 7 17:17 tube_n_ecrit.c
-rw-r--r-- 1 lovelace lovelace 497 Apr 7 17:21 tube_n_lect.c
...
$

```

Tubes nommés – Exemple d'écriture

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

int main(int argc, char * argv[])
{
    char c;
    int tube_nomme;
    printf("*** début du processus écriture dans le tube nommé\n");
    if (argc!=2)
    {
        fprintf(stderr,"Usage : %s nom_tube\n",argv[0]);
        exit(1);
    }
    if ((tube_nomme = open(argv[1], O_WRONLY)) == -1)
    {
        perror(argv[1]);
        exit(2);
    }
    printf("Le tube nommé %s a été ouvert \n", argv[1]);
    while (read (STDIN_FILENO, &c, sizeof(c))>0)
        write(tube_nomme, &c, sizeof(c));
    close(tube_nomme);
    exit(0);
}
```

Ouverture en écriture du tube nommé
Son nom est en paramètre
de la fonction main

Lecture de caractères au clavier
et écriture de ces caractères dans le tube

```
#include <stdio.h>
...
#include <sys/stat.h>
#include <fcntl.h>

int main(int argc, char * argv[])
{
    char c;
    int tube_nomme;
    printf("*** Début du processus lecture dans le tube nommé\n");
    if (argc!=2)
    {
        fprintf(stderr,"Usage : %s nom_tube\n",argv[0]);
        exit(1);
    }
    if ((tube_nomme = open(argv[1], O_RDONLY)) == -1)
    {
        perror(argv[1]);
        exit(2);
    }
    printf("Le tube nommé %s a été ouvert \n", argv[1]);
    while (read(tube_nomme, &c, sizeof(c)))
        write(STDOUT_FILENO, &c, sizeof(c));
    close(tube_nomme);
    exit(0);
}
```

Ouverture en écriture du tube nommé
Son nom est en paramètre
de la fonction main

Lecture des caractères dans le tube
et écriture de ces caractères dans la
sortie standard (STDOUT)

Lancement du processus « Producteur »
ERREUR

Le tube n'existe pas (le tube créé
précédemment a été supprimé)

```
$ ./ex_tube_e TN  
*** Début du processus écriture dans le tube nommé  
TN : No such file or directory  
$
```

```

$ ./ex_tube_e TN
*** Début du processus écriture dans le tube nommé
TN : No such file or directory
$
  
```

Création du tube nommé
Son nom est en paramètre
de la fonction main

```

$ ./ex_mkfifo TN
*** Début du processus création tube nommé
Le tube nommé TN a été créé
$ ls -l
total 384
prw-----  1 lovelace  lovelace      0 Apr  7 17:34 TN
-rwxr-xr-x  1 lovelace  lovelace  17380 Apr  7 09:37 ex_exec
...
$
  
```

Les droits d'accès précisent
lecture et écriture pour
le propriétaire seul

Tubes nommés – Exécution de l'exemple

```

$ ./ex_tube_e TN
*** Début du processus écriture dans le tube nommé
TN : No such file or directory
$
  
```

```

$ ./ex_mkfifo TN
*** Début du processus création tube nommé
Le tube nommé TN a été créé
$ ls -l
total 384
prw----- 1 lovelace lovelace 0 Apr 7 17:34 TN
-rwxr-xr-x 1 lovelace lovelace 17380 Apr 7 09:37 ex_exec
...
$
  
```

Lancement du processus « Producteur »

```

$ ./ex_tube_e TN
*** Début du processus écriture dans le tube nommé
Le tube nommé TN a été ouvert
a
sdfg
$
  
```

Le producteur lit les caractères au clavier et les écrit dans le tube

Lancement du processus « Consommateur »

```

$ ./ex_tube_l TN
*** Début du processus lecture dans le tube nommé
Le tube nommé TN a été ouvert
a
sdfg
$
  
```

Le consommateur lit les caractères dans le tube et les écrit à l'écran

```

$ ./ex_tube_e TN
*** Début du processus écriture dans le tube nommé
TN : No such file or directory
$
  
```

```

$ ./ex_mkfifo TN
*** Début du processus création tube nommé
Le tube nommé TN a été créé
$ ls -l
total 384
prw----- 1 lovelace lovelace 0 Apr 7 17:34 TN
-rwxr-xr-x 1 lovelace lovelace 17380 Apr 7 09:37 ex_exec
  
```

```

$ ./ex_tube_e TN
*** Début du processus écriture dans le tube nommé
Le tube nommé TN a été ouvert
a
sdfg
$
  
```

```

$ ./ex_tube_l TN
*** Début du processus lecture dans le tube nommé
Le tube nommé TN a été ouvert
a
sdfg
$
  
```

```

$ rm TN
$
  
```

Le tube nommé est un fichier permanent ;
il doit être détruit après utilisation